13. Buffers

Getting Started With SuperCollider

See also: 00. Getting Started With SC

Buffers represent server buffers, which are ordered arrays of floats on the server. 'float' is short for floating point number, which means a number with a decimal point, like 1.3. This is in contrast to integers, which are positive or negative whole numbers (or zero), and are written without decimal points. So 1 is an integer, but 1.0 is a float.

Server buffers can be single or multichannel, and are the usual way of storing data server-side. Their most common use is to hold soundfiles in memory, but any sort of data that can be represented by floats can be stored in a buffer.

Like busses, the number of buffers is set before you boot a server (using ServerOptions), but before buffers can be used, you need to allocate memory to them, which is an asynchronous step. Also like busses, buffers are numbered, starting from 0. Using Buffer takes care of allocating numbers, and avoids conflicts.

You can think of buffers as the server-side equivalent of an Array, but without all the elegant OOP functionality. Luckily with Buffer, and the ability to manipulate data in the client app when needed, you can do almost anything you want with buffer data. A server's buffers are global, which is to say that they can be accessed by any synth, and by more than one at a time. They can be written to or even changed in size, *while* they are being read from.

Many of Buffer's methods have numerous arguments. Needless to say, for full information see the Buffer help file.

Making a Buffer Object and Allocating Memory

Making a Buffer object and allocating the necessary memory in the server app is quite easy. You can do it all in one step with Buffer's alloc method:

```
s.boot;
b = Buffer.alloc(s, 100, 2);    // allocate 2 channels, and 100 frames
b.free;                 // free the memory (when you're finished using it)
```

The example above allocates a 2 channel buffer with 100 frames. The actual number of values stored is numChannels * numFrames, so in this case there will be 200 floats. So each frame is in this case a pair of values.

If you'd like to allocate in terms of seconds, rather than frames, you can do so like this:

```
b = Buffer.alloc(s, s.sampleRate * 8.0, 2); // an 8 second stereo buffer
b.free;
```

Buffer's 'free' method frees the memory on the server, and returns the Buffer's number for reallocation. You should not use a Buffer object after doing this.

Using Buffers with Sound Files

Buffer has another class method called 'read', which reads a sound file from disk into memory, and returns a Buffer object. Using the UGen PlayBuf, we can play the file.

```
// read a soundfile
b = Buffer.read(s, Platform.resourceDir +/+ "sounds/a11wlk01.wav");

// now play it
(
x = SynthDef("tutorial-PlayBuf",{ arg out = 0, bufnum;
    Out.ar( out,
        PlayBuf.ar(1, bufnum, BufRateScale.kr(bufnum))
    )
```

```
x.free; b.free;
```

PlayBuf.ar has a number of arguments which allow you to control various aspects of how it works. Take a look at the PlayBuf helpfile for details of them all, but for now lets just concern ourselves with the first three, used in the example above.

```
PlayBuf.ar(
    1,                // number of channels
    bufnum,             // number of buffer to play
    BufRateScale.kr(bufnum)        // rate of playback
    )
```

Number of channels: When working with PlayBuf you must let it know how many channels any buffer it will read in will have. *You cannot make this an argument in the SynthDef and change it later.* Why? Remember that SynthDefs must have a fixed number of output channels. So a one channel PlayBuf is always a one channel PlayBuf. If you need versions that can play varying numbers of channels then make multiple SynthDefs or use Function-play.

Buffer Number: As noted above, Buffers are numbered, starting from zero. You can get a Buffer's number using its **bufnum** method, but you will not normally need to do this, since Buffer objects can be passed directly as UGen inputs or Synth args.

Rate of Playback: A rate of 1 would be normal speed, 2 twice as fast, etc. But here we see a UGen called BufRateScale. What this does is check the samplerate of the buffer (this is set to correspond to that of the soundfile when it is loaded) and outputs the rate which would correspond to normal speed. This is useful because the soundfile we loaded (a11wlk01.wav) actually has a samplerate of 11025 Hz. With a rate of 1, PlayBuf would play it back using the sampling rate of the server, which is usually 44100 Hz, or four times as fast! BufRateScale thus brings things back to normal.

Streaming a File in From Disk

In some cases, for instance when working with very large files, you might not want to load a sound completely into memory. Instead, you can stream it in from disk a bit at a time, using the UGen DiskIn, and Buffer's 'cueSoundFile' method:

```
(
SynthDef("tutorial-Buffer-cue",{ arg out=0,bufnum;
    Out.ar(out,
        DiskIn.ar( 1, bufnum )
    )
}).add;
)

b = Buffer.cueSoundFile(s,Platform.resourceDir +/+ "sounds/a11wlk01-44_1.aiff", 0,
1);
y = Synth.new("tutorial-Buffer-cue", [\bufnum,b], s);

b.free; y.free;
```

This is not as flexible as PlayBuf (no rate control), but can save memory. [1]

More on Instance Variables and Action Functions

Now a little more OOP. Remember that individual Objects store data in *instance variables*. Some instance variables have what are called getter or setter methods, which allow you to get or set their values. We've already seen this in action with Buffer's 'bufnum' method, which is a getter for its buffer number instance variable.

Buffer has a number of other instance variables with getters which can provide helpful information. The ones we're interested in at the moment are numChannels, numFrames, and sampleRate. These can be particularly useful when working with sound files, as we may not have all this information at our fingertips before loading the file.

```
```

```
b.bufnum;
b.numFrames;
b.numChannels;
b.sampleRate;
b.free;
```

Now (like with the example using an action function in our Bus-get example; see 11. Busses) because of the small messaging latency between client and server, instance variables will not be immediately updated when you do something like read a file into a buffer. For this reason, many methods in Buffer take action functions as arguments. Remember that an action function is just a Function that will be evaluated after the client has received a reply, and has updated the Buffer's vars. It is passed the Buffer object as an argument.

```
// with an action function
// note that the vars are not immediately up-to-date
(
b = Buffer.read(s, Platform.resourceDir +/+ "sounds/a11wlk01.wav", action: { arg
buffer;
    ("numFrames after update:" + buffer.numFrames).postln;
    x = { PlayBuf.ar(1, buffer, BufRateScale.kr(buffer)) }.play;
});

// Note that the next line will execute BEFORE the action function
("numFrames before update:" + b.numFrames).postln;
)
x.free; b.free;
```

In the example above, the client sends the read command to the server app, along with a request for the necessary information to update the Buffer's instance variables. It then cues the action function to be executed when it receives the reply, and continues executing the block of code. That's why the 'Before update...' line executes first.

Recording into Buffers

In addition to PlayBuf, there's a UGen called RecordBuf, which lets you record into a buffer.

```
b = Buffer.alloc(s, s.sampleRate * 5, 1); // a 5 second 1 channel Buffer

// record for four seconds
(
x = SynthDef("tutorial-RecordBuf",{ arg out=0,bufnum=0;
    var noise;
    noise = PinkNoise.ar(0.3);    // record some PinkNoise
    RecordBuf.ar(noise, bufnum);     // by default this loops
}).play(s,[\out, 0, \bufnum, b]);
)

// free the record synth after a few seconds
x.free;

// play it back
(
SynthDef("tutorial-playback",{ arg out=0,bufnum=0;
    var playbuf;
    playbuf = PlayBuf.ar(1,bufnum);
    FreeSelfWhenDone.kr(playbuf); // frees the synth when the PlayBuf has played
through once
    Out.ar(out, playbuf);
}).play(s,[\out, 0, \bufnum, b]);
)
```

See the RecordBuf help file for details on all of its options.

Accessing Data

Buffer has a number of methods to allow you to get or set values in a buffer. Buffer-get and Buffer-set are straightforward to use and take an index as an argument. Multichannel buffers interleave their data, so for a two channel buffer index 0 = frame1-chan1, index 1 = frame1-chan2, index 2 = frame2-chan1, and so on. 'get' takes an action function.

```
b = Buffer.alloc(s, 8, 1);
b.set(7, 0.5);              // set the value at 7 to 0.5
b.get(7, {|msg| msg.postln});    // get the value at 7 and post it when the reply
is received
b.free;
```

The methods 'getn' and 'setn' allow you to get and set ranges of adjacent values. 'setn' takes a starting index and an array of values to set, 'getn' takes a starting index, the number of values to get, and an action function.

```
b = Buffer.alloc(s,16);
b.setn(0, [1, 2, 3]);                // set the first 3 values
b.getn(0, 3, {|msg| msg.postln});        // get them
b.setn(0, Array.fill(b.numFrames, {1.0.rand}));    // fill the buffer with random
values
b.getn(0, b.numFrames, {|msg| msg.postln});    // get them
b.free;
```

There is an upper limit on the number of values you can get or set at a time (usually 1633 when using UDP, the default). This is because of a limit on network packet size. To overcome this Buffer has two methods, 'loadCollection' and 'loadToFloatArray' which allow you to set or get large amounts of data by writing it to disk and then loading to client or server as appropriate.

```
(
// make some white noise
v = FloatArray.fill(44100, {1.0.rand2});
b = Buffer.alloc(s, 44100);
)
(
// load the FloatArray into b, then play it
b.loadCollection(v, action: {|buf|
    x = { PlayBuf.ar(buf.numChannels, buf, BufRateScale.kr(buf), loop: 1)
        * 0.2 }.play;
});
)
x.free;

// now get the FloatArray back, and compare it to v; this posts 'true'
// the args 0, -1 mean start from the beginning and load the whole buffer
b.loadToFloatArray(0, -1, {|floatArray| (floatArray == v).postln });
b.free;
```

A FloatArray is just a subclass of Array which can only contain floats.

Plotting and Playing

Buffer has two useful convenience methods: 'plot' and 'play'.

```
// see the waveform
b = Buffer.read(s, Platform.resourceDir +/+ "sounds/a11wlk01.wav");
b.plot;
```

```
// this takes one arg: loop. If false (the default) the resulting synth is
// freed automatically
b.play;             // frees itself
x = b.play(true);        // loops so doesn't free
x.free; b.free;
```

Other Uses For Buffers

In addition to being used for loading in sound files, buffers are also useful for any situation in which you need large and/or globally accessible data sets on the server. One example of another use for them is as a lookup table for waveshaping.

```
b = Buffer.alloc(s, 512, 1);
b.cheby([1,0,1,1,0,1]);
(
x = play({
    Shaper.ar(
        b,
        SinOsc.ar(300, 0, Line.kr(0,1,6)),
        0.5
    )
});
)
x.free; b.free;
```

The Shaper UGen performs waveshaping on an input source. The method 'cheby' fills the buffer with a series of chebyshev polynomials, which are needed for this. (Don't worry if you don't understand all this.) Buffer has many similar methods for filling a buffer with different waveforms.

There are numerous other uses to which buffers can be put. You'll encounter them throughout the documentation.

For more information see:

Buffer, PlayBuf, RecordBuf, SynthDef, BufRateScale, Shaper

This document is part of the tutorial **Getting Started With SuperCollider**.

Click here to go on to the next section: 14. Scheduling Events

Click here to return to the table of Contents: 00. Getting Started With SC

---

[1] - For variable playback rate while streaming from disk, look at the VDiskIn UGen.